Queen Mary University of London

Introduction to Python for Data Science and Machine Learning

Nikesh Bajaj, PhD
Lecturer in Data Science, Director of Education
Queen Mary University of London
nikesh.bajaj@qmul.ac.uk
https://nikeshbajaj.in

Outline

- Programming Languages and Tools
- Introduction to Python
- Lab activities:
  - 1. Python Starter
  - 2. EEG: reading files, and visualisation
  - 3. EEG: Topographical maps

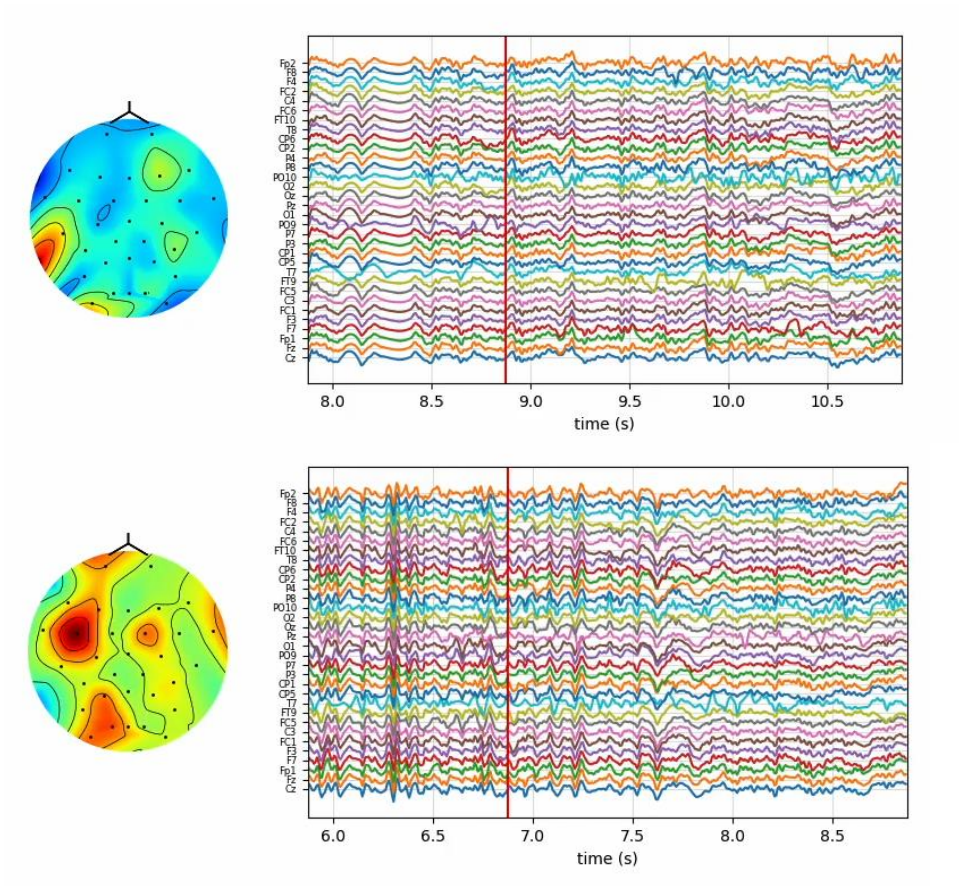
We will cover a very condensed version of tutorials for Python, more detailed slides can be found at https://nikeshbajaj.github.io/teaching/QMUL/QHP4701/

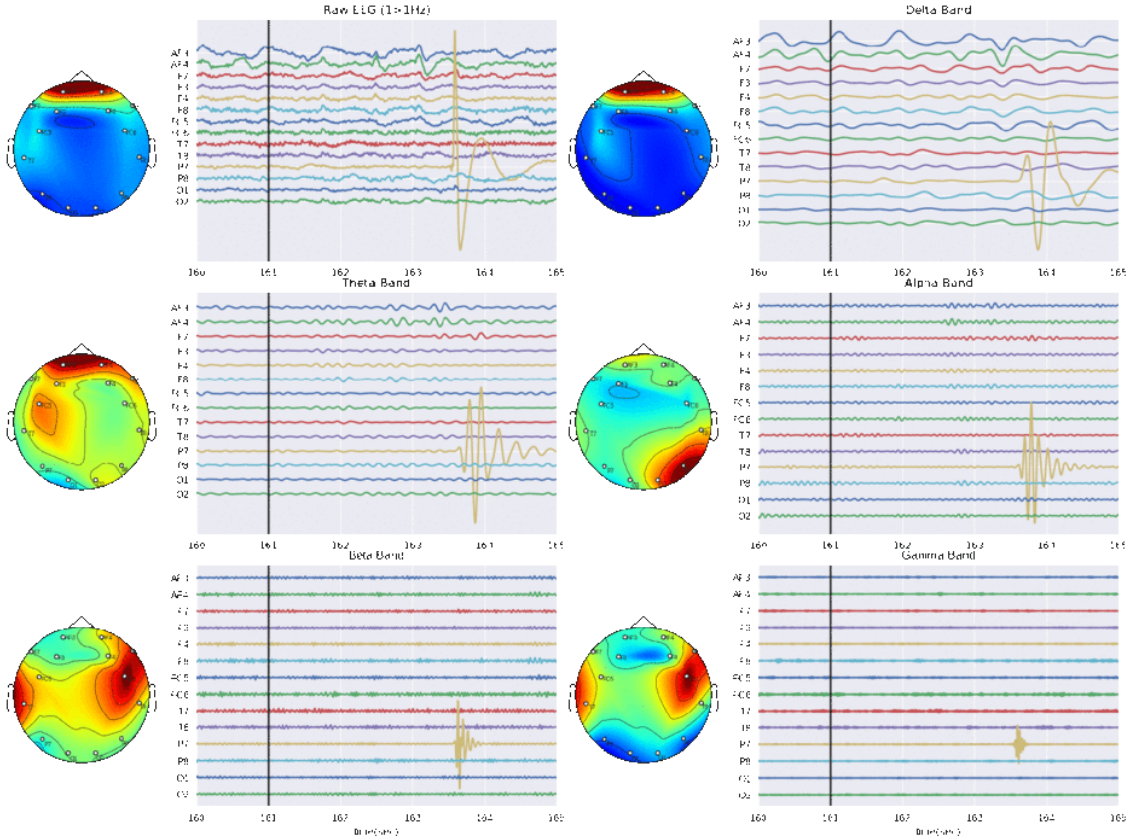
***Other Book**: Ref: Python Data Science Handbook, 2nd Edition,*

Link: https://jakevdp.github.io/PythonDataScienceHandbook/

Lab activity includes

- Reading, manipulation EEG recording
- Plot some cool visualisations.



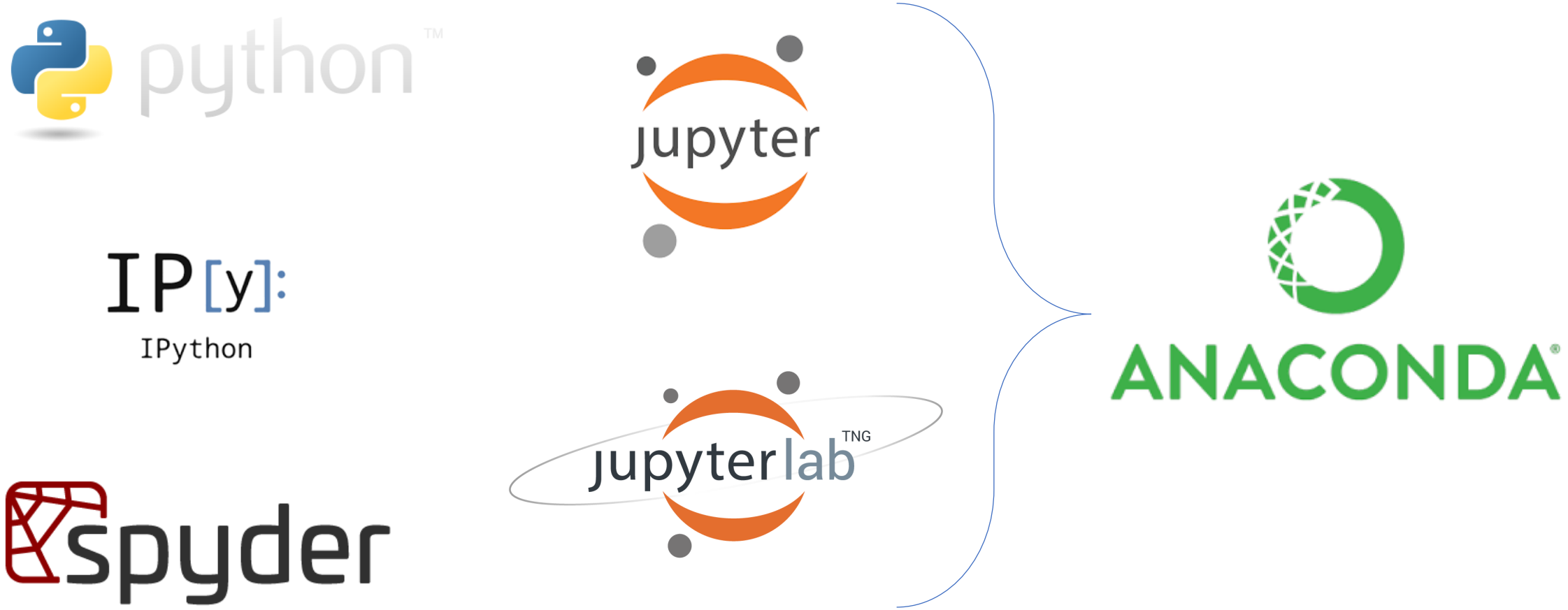


Programming Languages and Tools for Data Science

- MATLAB
- R (Studio)
- **Python (popular choice)**
- Apache Hadoop
- Apache Spark
- SQL
- Docker
- Azure
- .
- … more

Programming in the module

We will be using **Python**, and particularly:



What is Python & Anaconda

Python

- Python is a general purpose, object-oriented, high-level programming language. It was created by Guido van Rossum (1991). It is used for software development, scripting, mathematics and very popular for Data Science Development and Research

Anaconda

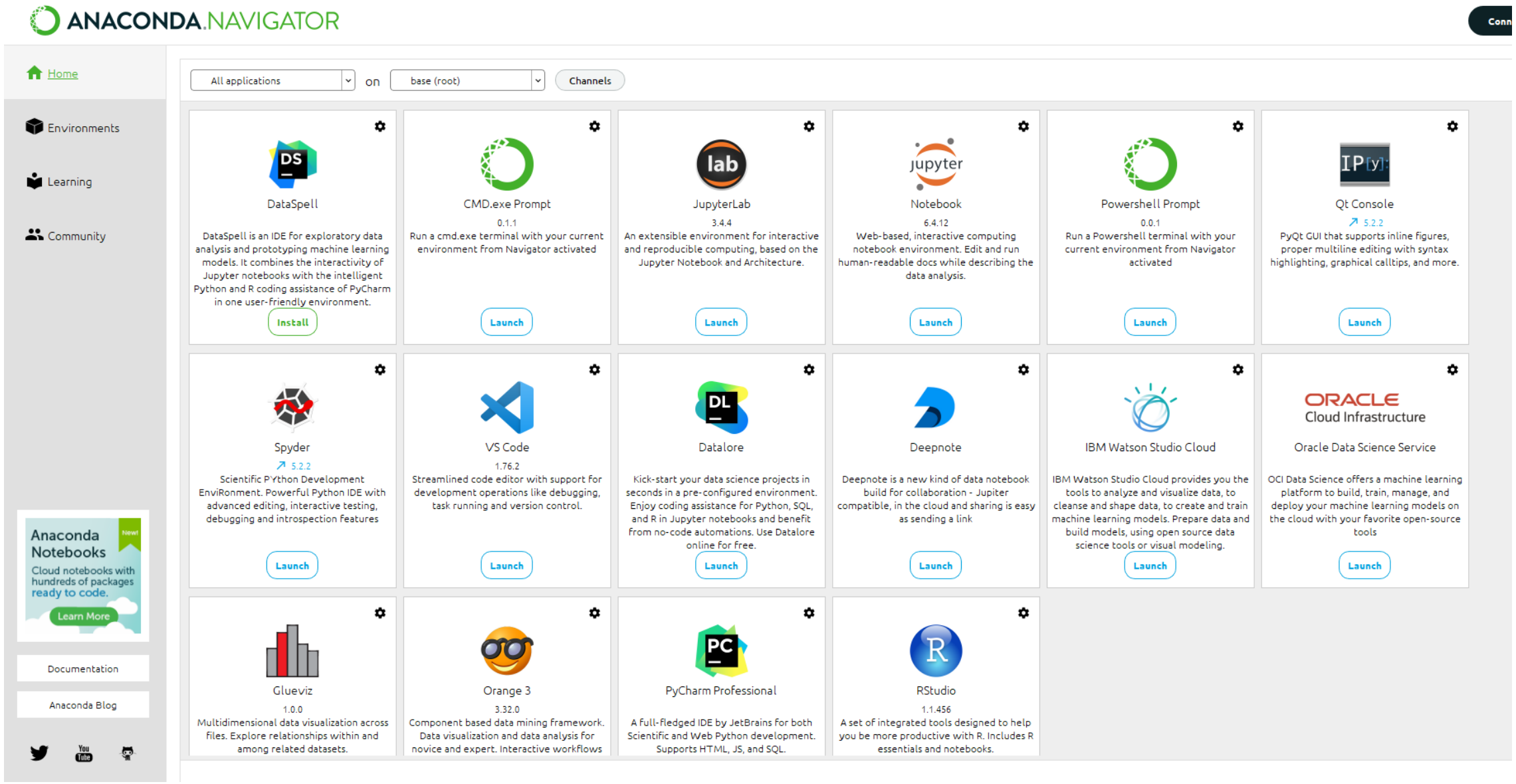
- Anaconda is a distribution of many programming languages like Python and R and tools. It uses an open-source package and environment management system called ***Conda***, which makes it easy to install/update packages and manage environments.

Why Python?

- **Open source**
  - Freely available to use
- A large community:
  - Many people around the world use it, can help resolving issues
- Simple, easy and very powerful language
  - Can handle large amount of data, integrate with different HW, Software, Web, etc.

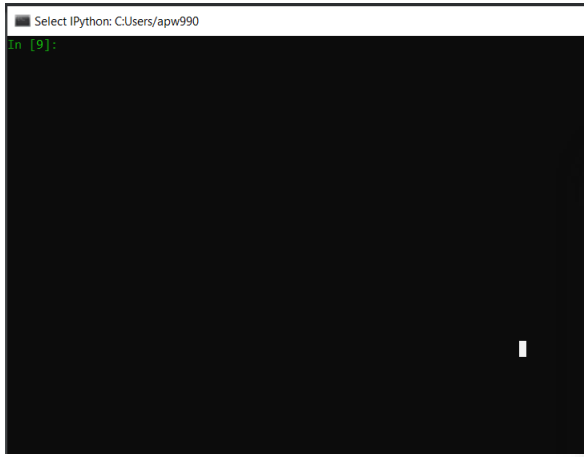
- For Data Science
  - Large number of tools for data science are freely available,
  - Any development of tool/algorithm/model can be integrated to other systems
  - Nicer visualization, documentation, and Reports can be build (Jupyter-notebook)

A walk through Anaconda

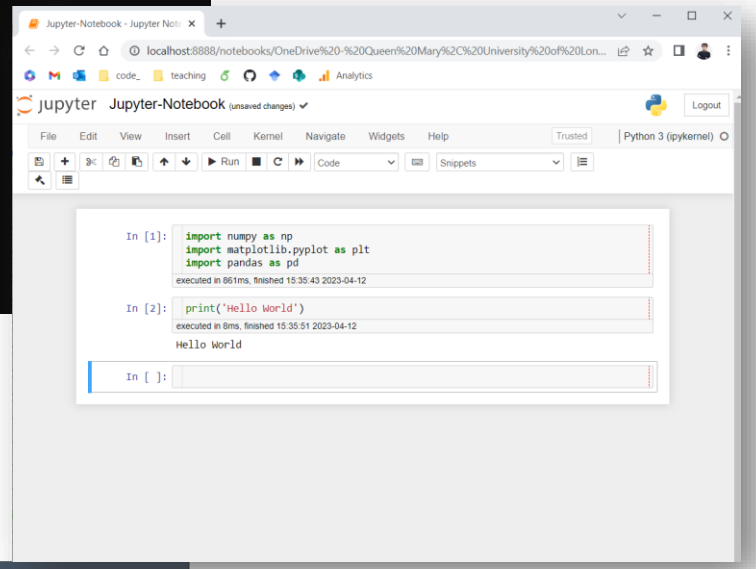


Python: Interfaces

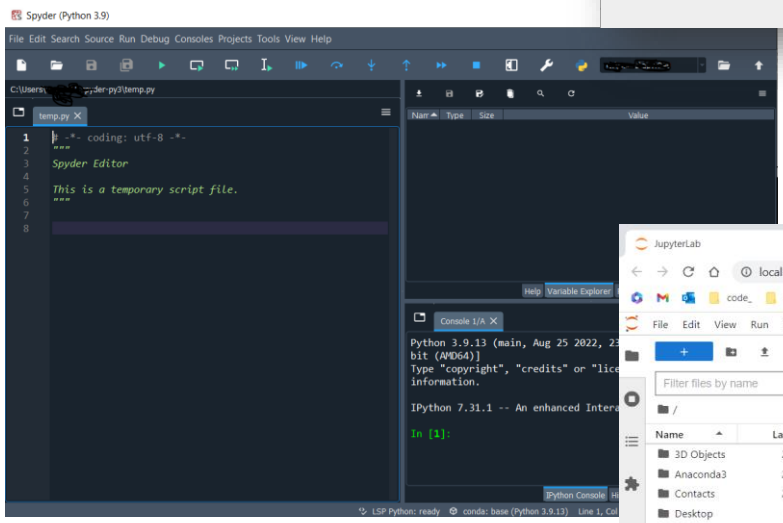
- IPython Terminal
  - type: ‘ipython’ on terminal (any OS)



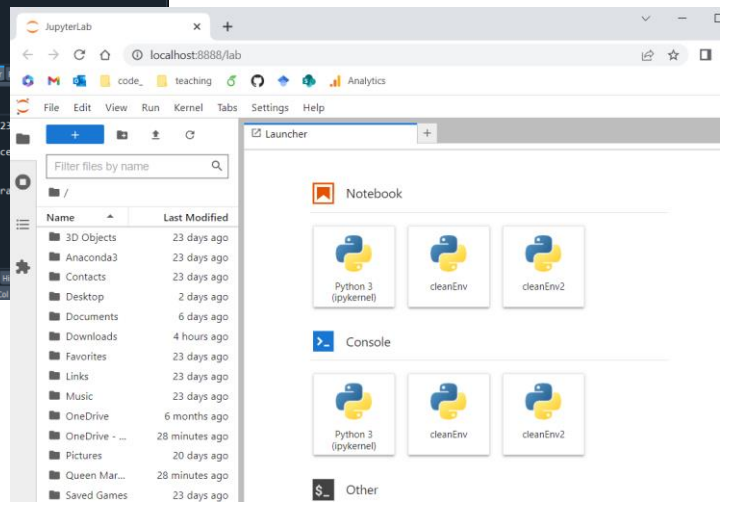
- Jupyter-Notebook:
  - type: ‘jupyter-notebook’ on anaconda terminal



- Spyder
  - type: ‘spyder’ on anaconda terminal



- **Jupyter-Lab**
  - type: ‘jupyter-lab’ on anaconda terminal



All can be opened from Anaconda Navigator

Python: As a calculator

- IPython –terminal
  - type: ‘ipython’ on terminal (any OS)

- 344+344
- 34-5
- 2345/232
- 234*4354
- 355 + (434*234) – (233-2)

- - need to create variables…??

Python: Arithmetic + use of variable

Use of variables: x, y, z

```
In [2]:
x = 4
y = 10

z1 = x + y
z2 = y - x
z3 = x * y
z4 = x / y
z5 = y // 2
z6 = x**2
z7 = y%x
```

executed in 9ms, finished 12:26:59 2023-03-

```
In [4]:
print(z1)
print(z2)
print(z3)
print(z4)
print(z5)
print(z6)
print(z7)
```

executed in 6ms, finished 12:28:39 2023-03-28

```
14
6
40
0.4
5
16
2
```

Python operation	Arithmetic operator	Algebraic expression	Python expression
Addition	+	$f + 7$	`f + 7`
Subtraction	–	$p - c$	`p - c`
Multiplication	*	$b \cdot m$	`b * m`
Exponentiation	**	x^y	`x ** y`
True division	/	x/y or $\frac{x}{y}$ or $x \div y$	`x / y`
Floor division	//	$\lfloor x/y \rfloor$ or $\left\lfloor \frac{x}{y} \right\rfloor$ or $\lfloor x \div y \rfloor$	`x // y`
Remainder (modulo)	%	$r \bmod s$	`r % s`

```
>>> y = (a * (x ** 2)) + (b * x) + c
>>> print(y)
```

What happens?

x/0
y/0

Python: Basics Data Types

How do you check Data Type of given variable x?

>>> type(x)

- Basic

  - **Numerical**: Integers (int), fractions (float) complex
  - **Strings**: str
  - **Boolean** True/False
  - **NoneType** None

  Objects

- Examples

  x = 23
  y = 45
  z = 1010
  x = 24.5023
  y = 0.0113
  z = 1.0
  x = 2 + i1

Name	Age	Height (feets)	Weight (Kg)	Address	Phone number	Number of languages known
Steve Johnson	21	5'6	55	21, Harrow	7475738232	2
John Smith	25	5'8	64	None	7847272382	1

  S1 = 'Hello World' or "Hello World"
  S2 = "H"

  x = True
  y = False
  x = None

Python: Data Types
Operations on Basics

How do you check Data Type of given variable x?

>>> type(x)

- Basic
  - Addition : x+y
  - Subtraction : x-y
  - Multiplication : x*y
  - Division : x/y
  - Power : x**y

For x:int or x:float, y:int or y:float

Exceptions: x/y for y=0

x = 4

y = ‘hello’

Multiplication :x*y

Only If one of variable is string and other in int

Addition : x+y

Only If both variables are string type

Collection of Data

- Basic Data Types hold a single value
- For recording sequence or collection of single values we need an efficient representation

Let's check an example from Table 1.1

H = 168, 160,169,170,168, 165,165, 170, 171, 168

P3 = 25, 169, 69.01, None, UG

PersonID	Age	Height (in cm)	Weight (in Kg)	Address	Education Level
P1	16	168	60.50	E16 3LW, Lon	Highschool
P2	21	160	61.30	E16 3LW, Lon	UG
P3	25	169	69.01	None	UG
P4	21	170	70.60	E16 3LW, Lon	UG
P5	23	168	59.10	E16 3LW, Lon	PG
P6	32	165	55.89	None	PhD
P7	None	165	59.00	E16 3LW, Lon	Highschool
P8	42	170	65.00	E16 3LW, Lon	Phd
P9	28	171	76.60	E16 3LW, Lon	PG
P10	27	168	79.90	E16 3LW, Lon	PG

Text – sequence of words, Image – grid of pixels, Speech – sequence of values - grid data

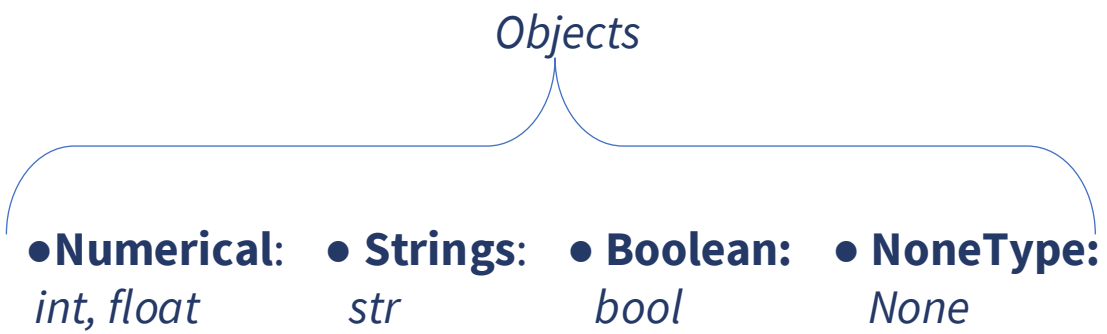
Collection of Data

Following Data Types in Python hold collection of values/objects

- List
- Tuple
- Set
- Dictionary

$$X = \left(\textit{Object} \quad \textit{Object} \quad \textit{Object} \quad \textit{Object} \right)$$

Collection of objects

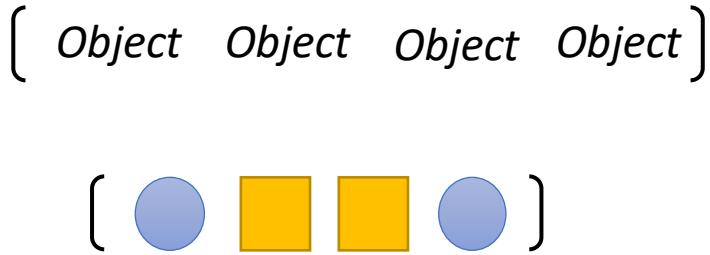


Each of them have different structure, purpose and operations, we will discuss each one by one

List

List

- Collection of elements (***objects*** or items)
- Most versatile and efficient data type
- Could be all of same data types or different



- Examples

[1, 2, 4, 10, -10]

['Apple', 'Oranges', 'Banana']

an element or an item

['A', 1, 'B', -19, None]

['P3', 25, 169, 69.01, '145, South London, SW12 LQ2, UK', None]

Operations on List

List

Creating a List

```
c = [-45, 6, 0, 72, 1543]
```

- Length of a List *len(c)*

- List of sequential numbers (integers)

c = list(range(5))

c = [0, 1, 2, 3, 4]

c = list(range(1,10,2))

c = [1,3,5,7,9]

range(end)

range(start, end)

range(start,end,step)

By default start is set to 0 and step is set to 1

Operations on List

Selecting Element(s)

- Indexing

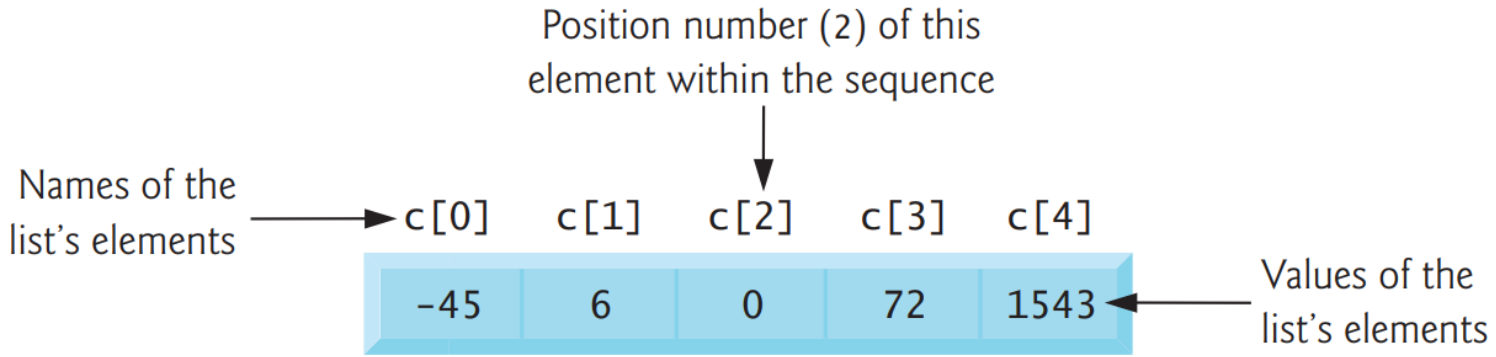
```
c = [-45, 6, 0, 72, 1543]
```

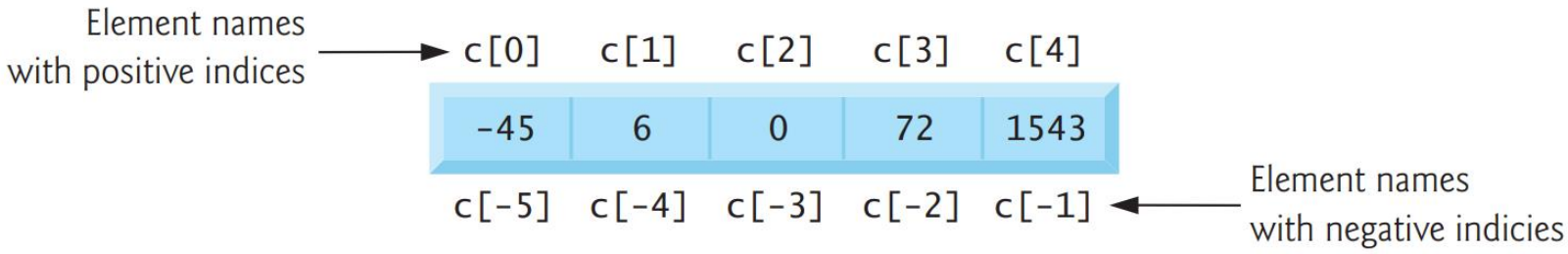
As most programming languages, Indexing in python starts with 0

C[0]

C[3]

C[-1]





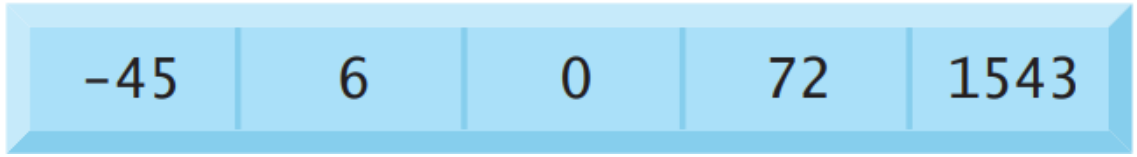
Operations on List

```
c = [-45, 6, 0, 72, 1543]
```

Selecting element(s):

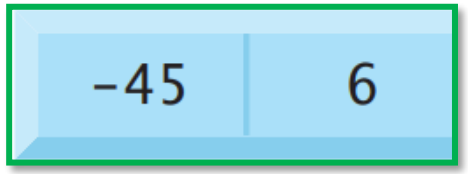
- Slicing

c =



c[1:3]

c[:2]



c[2:]

Try: c[-2:], c[::2], c[::-1]

Excluding

c[starts: end]
c[start:end:step]
c[::step]

'*List*' of methods

	.append()	
	.count()	2
	.insert(1, )	
	.remove()	
	.index()	2
	.pop(1)	
	.sort( )	
	.clear( )	
	.copy( )	
	.reverse( )	

Tuple and Immutability

Tuple and List share same structure. The key difference in tuple are immutable.

- a = (1,2,3)
  a[0] = 10

- b = (1,2,4, [1,2,4,5],None, ‘A’)
  b[3][0]=10

List has more methods than tuple

Python Data Type set

Sets

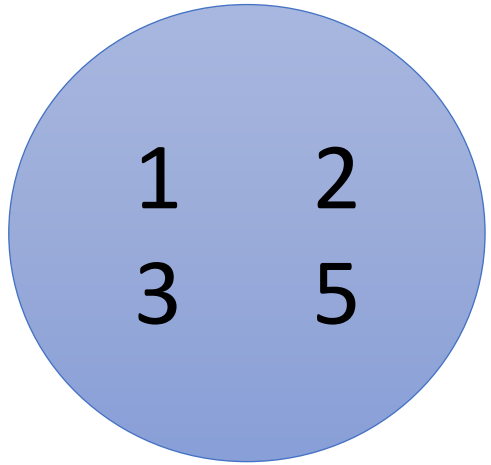
As defined in mathematics, a set is a collection of unique elements

Creating a set

- A = set([1, 2, 3, 3, 5])
- B = {1,2,3,3,5}

Adding /removing an element

- A.add(5)
- A.add(8)
- A.remove(1)
- A.discard(10)



A

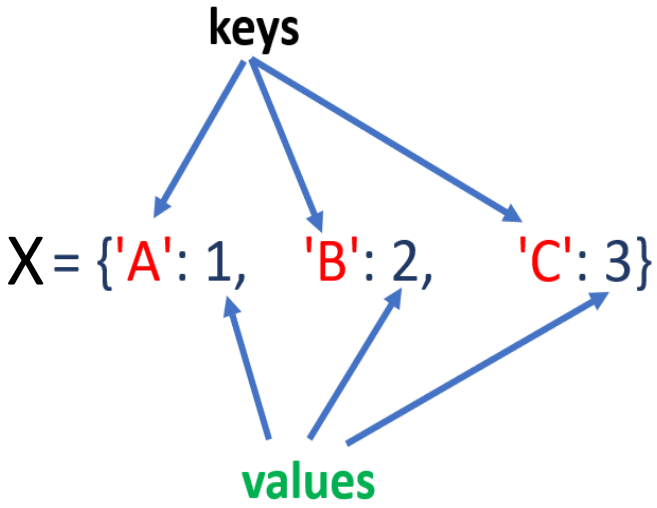
Dictionary : Mapping Type

- Dictionary in Python are great way to store data or any mapping pairs
- Dictionary has keys, values pair.

Three ways to create dictionary

- X = { 'A': 1, 'B':2, 'C':3}
- X = dict([('A',1), ('B',2), ('C',3)])
- X = dict(A=1, B=2,C=3)

A → 1
B → 2
C → 3



Keys in dictionary have to be unique

Data → [1,2,3,5,3,2,4]
photo → Image
Grades → ['A', 'A', 'B', 'A']

Mixing the collections

- List of Lists
- List of any type
  - List of tuples, dictionaries, sets or mix of them

- Tuple of Tuples
- Tuple of any type
  - tuple of lists, dictionaries, sets or mix of them

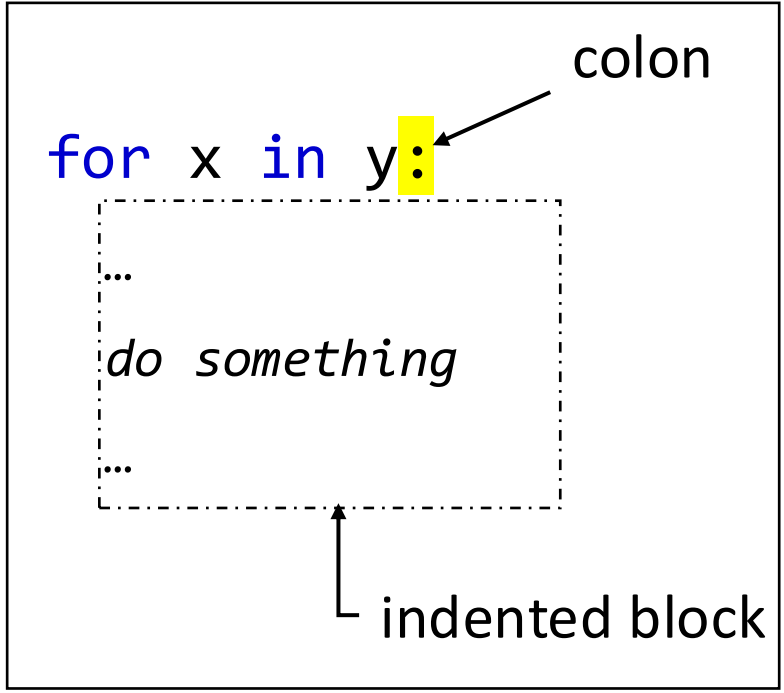
- Dictionary
  - keys, a list can not a key, but tuples can be
  - Anything can be in values

- Sets
  - Element of a set can be tuple but not list or set

Mixing of dict and set is not always straightforward

Loops: for-loop

- For-loop: In programming languages, for repeating operation(s), a loop is used, which iterate over a sequence (goes over each element of a sequence)




```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
  print(fruit)
```

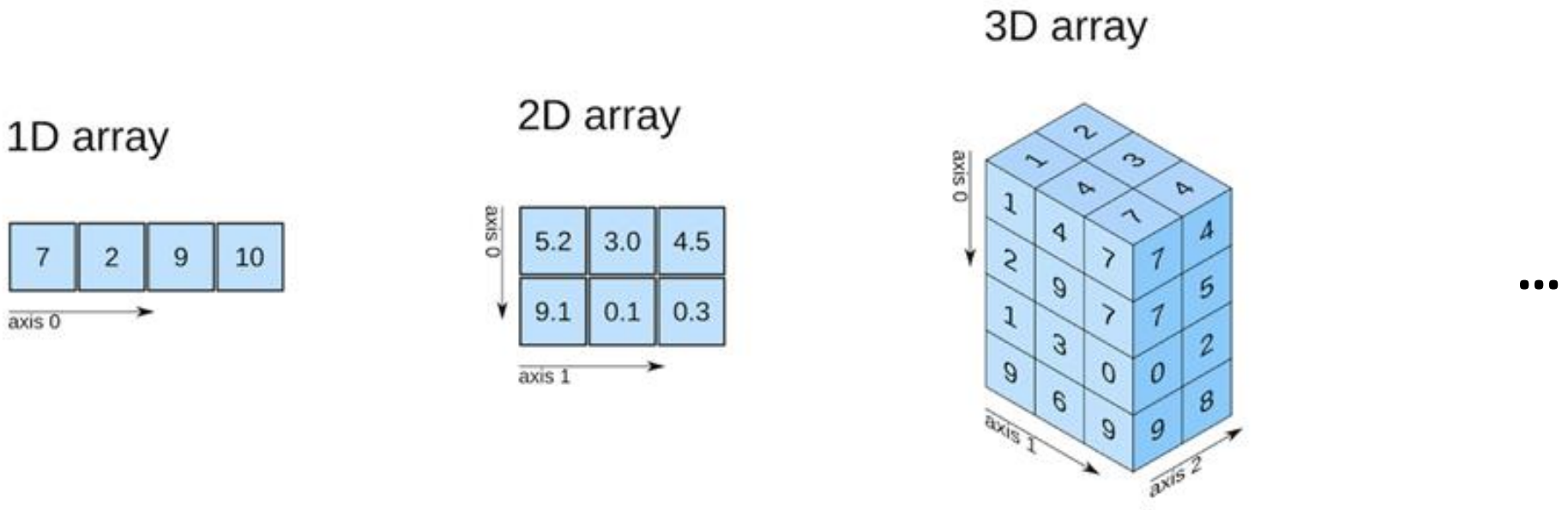
```
for i in range(6):
  print(i)
```

```
numbers = [1,2,3,4,5]
squares = []
for num in numbers:
    squares.append(num ** 2)

print(squares)
```

NumPy in Python

- NumPy in Python is a very basic library used for scientific computations. It is heavily used in Data Science.
- It provides arrays (vectors, matrices) as objects that supports linear algebra, sorting, selecting and manipulating data



...

NumPy in Python

- Anaconda comes with NumPy and several other libraries used for Data Science
- To use NumPy, first you we load the library

```
import numpy
```

Or

```
import numpy as np
```

NumPy : Creating Arrays

Creating Arrays from sequences

- x1D = np.array([1, 2, 3, 4])
- x2D = np.array([[1, 2],
  [3, 4]])
- x3D = np.array([[[1, 2], [3, 4]],
  [[5, 6], [7, 8]]])

Check type and share and size

- type(x1D)
- x1D.shape
- x2D.shape
- x3D.shape
- x3D.size

NumPy : Creating Arrays

Creating Arrays from functions

- x = np.arange(10)
- x = np.arange(2, 10, 2) # can accept float
- x = np.linspace(0,10,10)
- x = np.eye(3)
- x = np.diag([1,2,3])
- x = np.zeros([2,3])
- x = np.ones([2,3])
- x = np.random.rand(10)
- x = np.random.randn(10)
- x = np.random.randint(0, 5, 10)

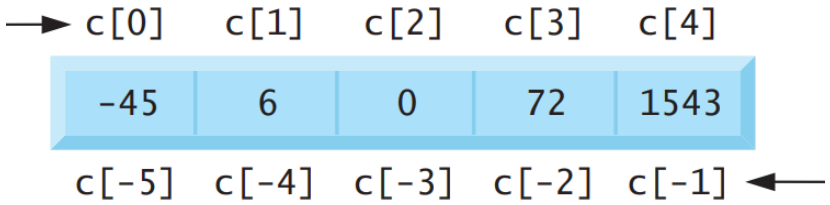
- Range from 0 to 9
- Range from 2 to 10 with step 2
- 10 values from 0 to 10 including 0 and 10 (evenly spaced)
- Identity matrix of 3x3
- Diagonal matrix
- Matrix of zeros
- Matrix of ones

Random 1d matrix of 10 values

NumPy : Indexing

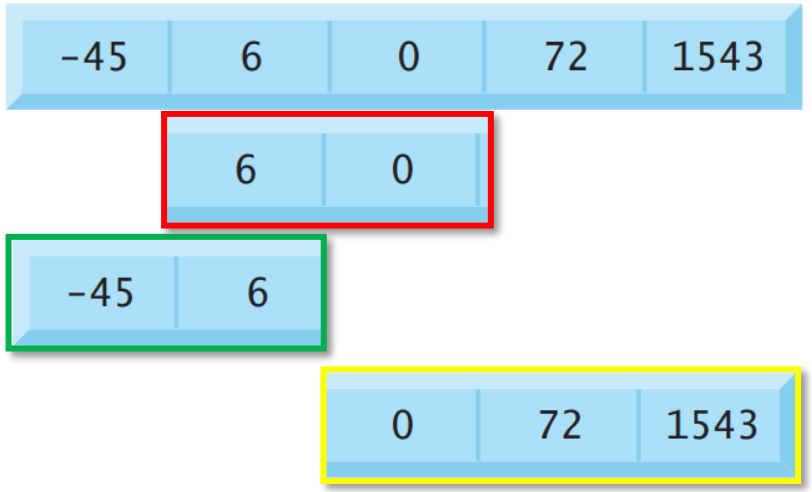
Indexing in Numpy Array is ***similar to list***

- c = np.array([-45, 6, 0, 72, 1543])



Selecting element(s):

- Slicing



Excluding

c[starts: end]
c[start:end:step]
c[::step]

NumPy : Linear Algebra

- Recall the Linear Algebra operations with List, NumPy make is very easy
- a = np.array([1,2,3,4,5])
- b = np.array([5,6,7,8,9])

$$a = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} \qquad b = \begin{bmatrix} 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{bmatrix}$$

- Addition: c = a+b
- Scalar multiplication c = 2*a
- Dot product c = a.dot(b)
- Squared c = a**2
- Magnitude c = sum(a**2)**(1/2) c = np.linalg.norm(a)

NumPy : Linear Algebra
Matrix Multiplication

- A = [[1, 2, 3],
  [2, 0, 1],
  [1, 0, 1]]

- B = [[2, 2, 3],
  [1, 1, 0],
  [2, 1, 0]]

Compute

- C = A × B

With NumPy:

- C = A@B
- C = A.dot(B)

NumPy : Methods and Operations

- a = np.array([1,2,3, 4,5,6])

$$a = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Methods

- sum, max, min a.sum(), a.max(), a.min() or np.sum(a), np.max(a) …
- mean, var, std, a.mean(), a.var(), a.std()

- **Compute for b**

$$b = \begin{bmatrix} 4 \\ 5 \\ 1 \\ 2 \\ 3 \\ 6 \end{bmatrix}$$

NumPy : Methods and Operations

$$a = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

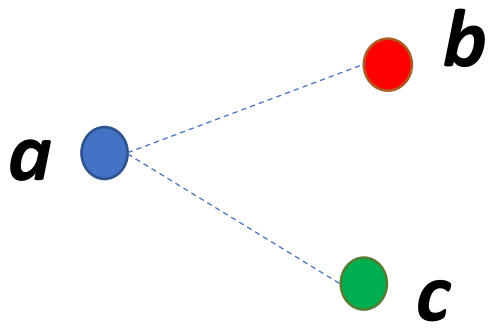
- np.add
- np.subtract
- np.multiply
- np.divid
- np.power, np.sqrt, np.square
- np.log, np.log2, np.log10
- np.exp
- np.abs, np.absolute
- np.sin, np.cos, np.tan, np.arcsin, np.arccos
- .
- ….
- *Ref: https://numpy.org/doc/stable/reference/ufuncs.html#available-ufuncs*

$$b = \begin{bmatrix} 4 \\ 5 \\ 1 \\ 2 \\ 3 \\ 6 \end{bmatrix}$$

NumPy : Euclidean Distance

- Euclidean distance between two points

$$D = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$



- Which one is closer to a?
  - b?
  - c?

$$\mathrm{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix} \qquad \mathrm{b} = \begin{bmatrix} 4 \\ 5 \\ 1 \\ 2 \\ 3 \\ 6 \end{bmatrix} \qquad \mathrm{c} = \begin{bmatrix} 4 \\ 4 \\ 3 \\ 3 \\ 3 \\ 6 \end{bmatrix}$$

Visualise with matplotlib

- In Python Matplotlib is used to visualise and plot different kind of data.
- Matplotlib will be covered in future session with great details.
- For now, to visualise arrays, we will use a few functionalities of matplotlib

```
import matplotlib.pyplot as plt
```

Additional Libraries to load data

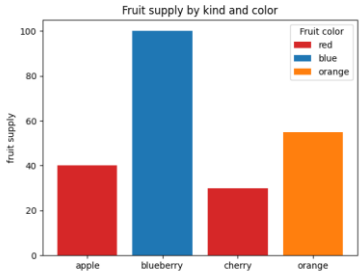
- For read different types of data, we need some additional libraries

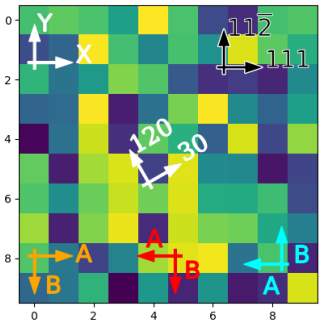
```
import pandas as pd
import matplotlib.pyplot as plt
import scipy
```

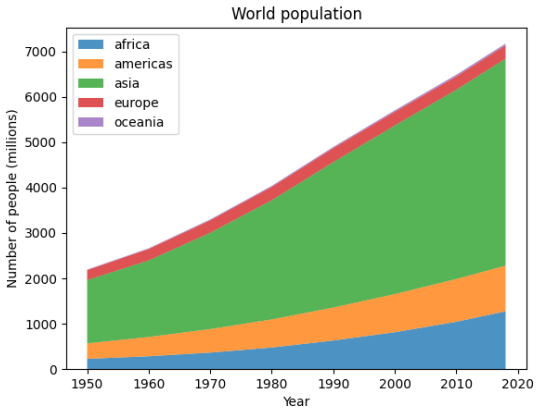
Audio/Speech	.wav	Scipy	*scipy.io.wavfile.read*
Image	.png, .bmp, .jpg	Matplotlib	*plt.imread*
CSV	.csv	Pandas	*pd.read_csv*

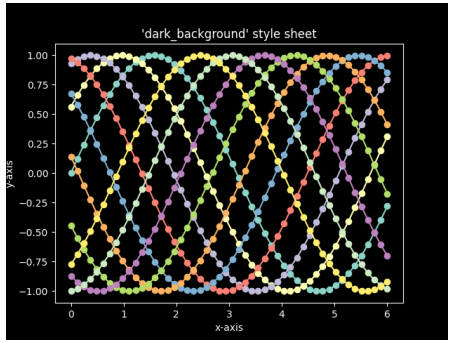
Matplotlib: Examples

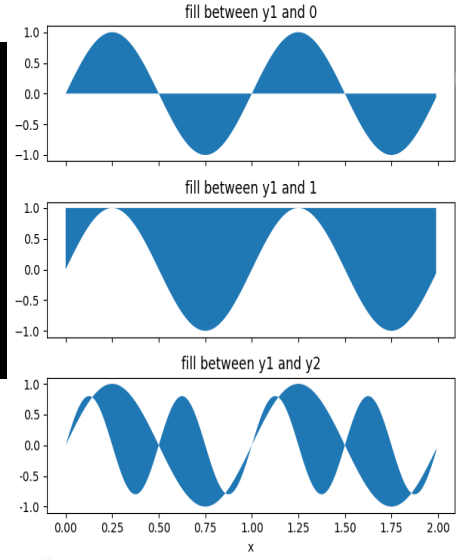
- Matplotlib allows you to visualise data in many different ways. Here are a few examples

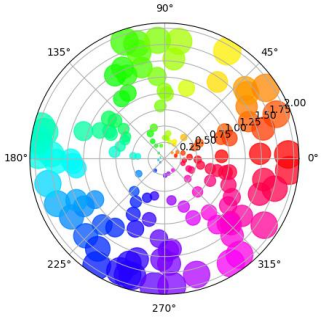


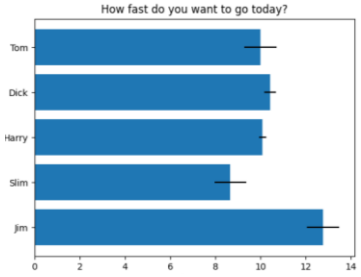


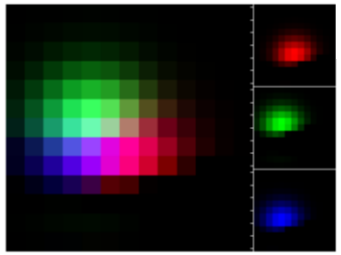


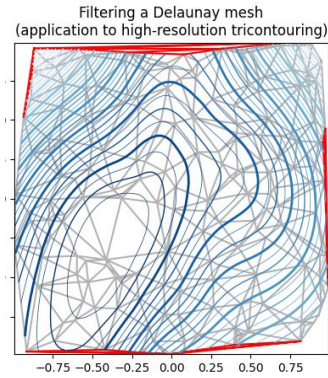


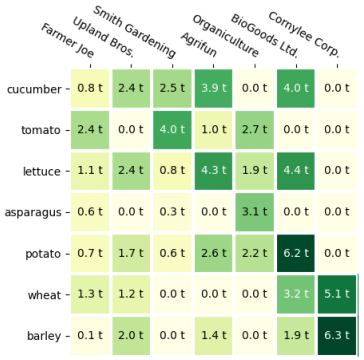


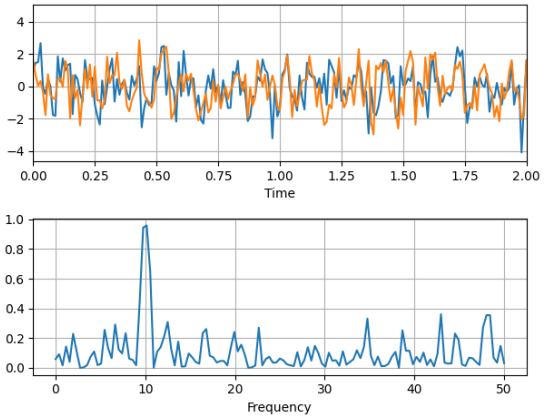


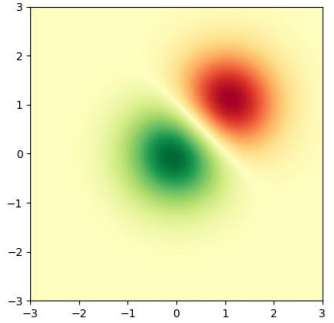


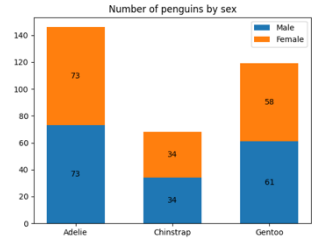


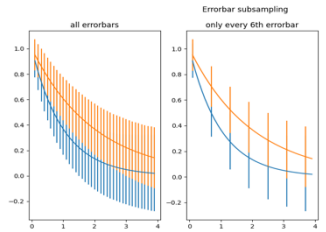


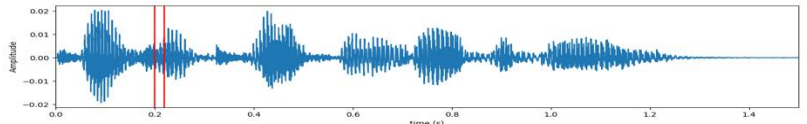










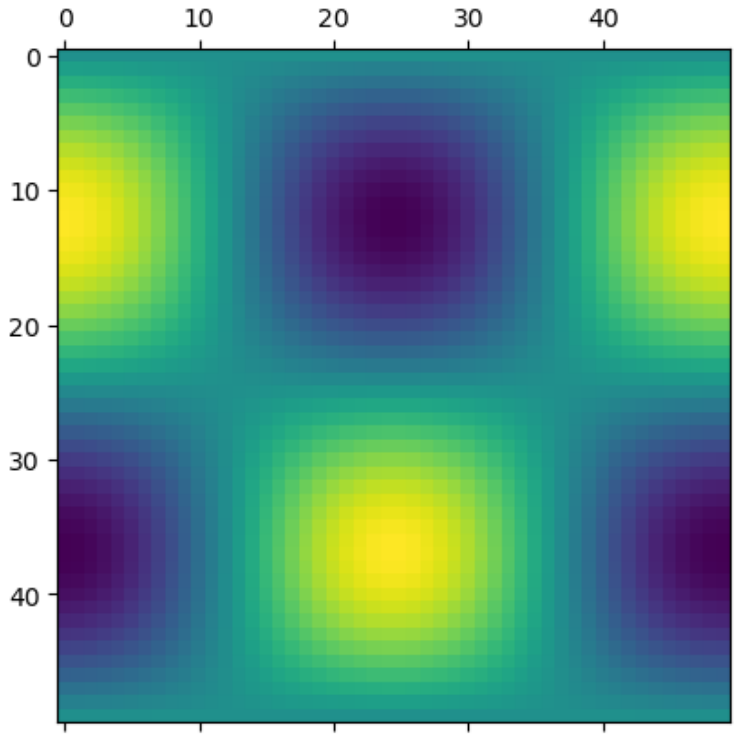


Heatmap, colormap

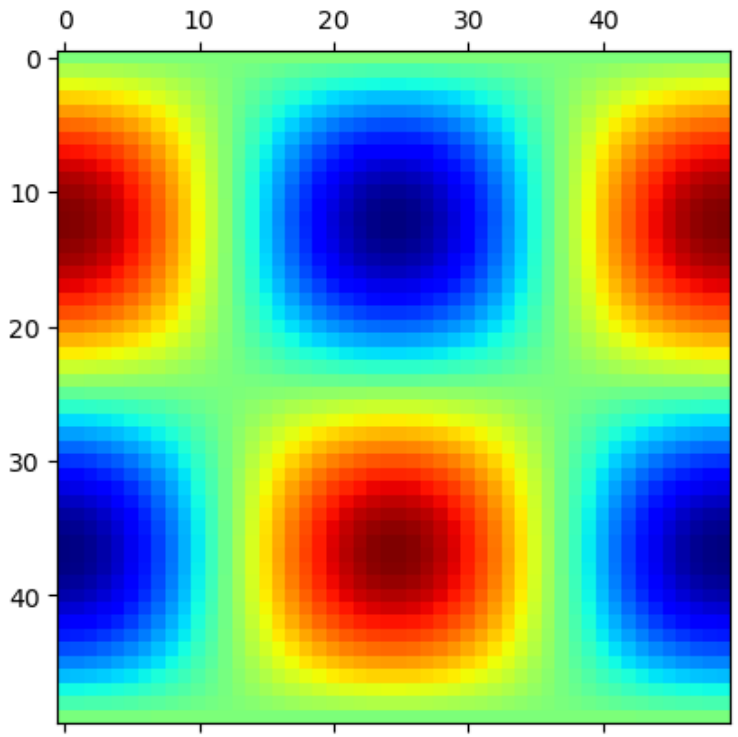
- colormap

```
t = np.linspace(0, 2, 50)
f = 0.5
x = np.sin(2*np.pi*f*t)
y = np.cos(2*np.pi*f*t)
z = x[:,None]@y[None,:]
```

```
plt.matshow(z)
plt.show()
```




```
plt.matshow(z, cmap='jet')
plt.show()
```



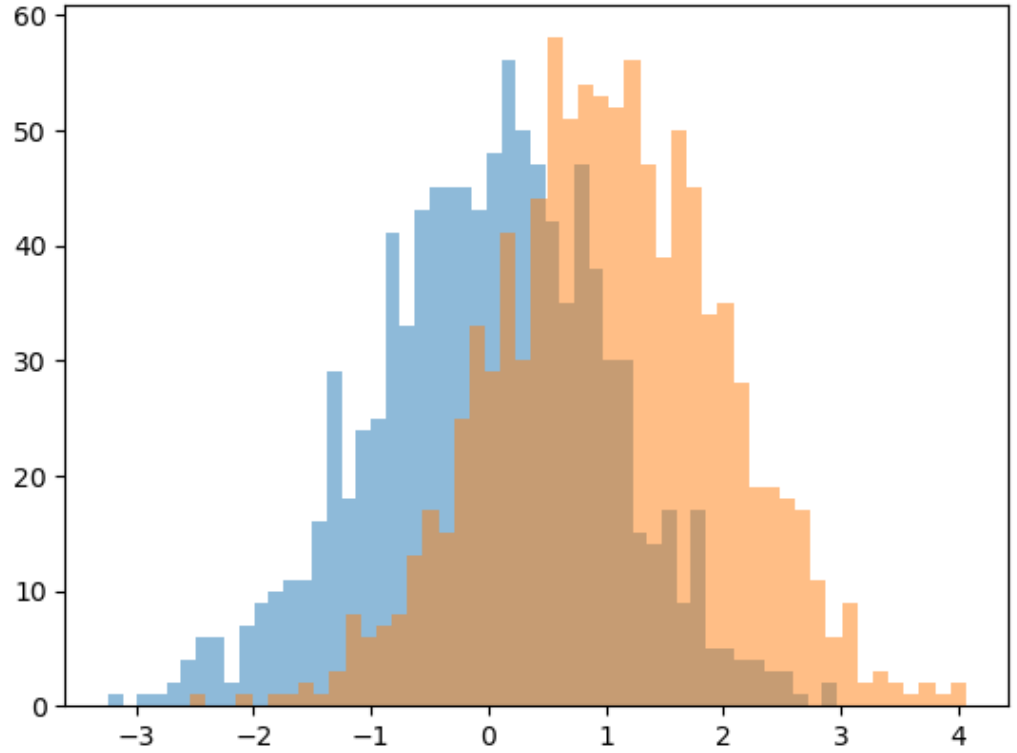
Decorating Figures

- Following

```
plt.title
plt.xlabel, plt.ylabel
plt.xlim, plt.ylim
plt.xticks, plt.yticks
Figure-size
Subplots
Color
Linestyle
Markertype
Markersize
Alpha
```

```
x = np.random.randn(1000)
y = np.random.randn(1000)+1

plt.figure()
plt.hist(x,alpha=0.5,bins=50)
plt.hist(y,alpha=0.5,bins=50)
plt.show()
```



More on Python:

Detailed slides for Python can be found at

https://nikeshbajaj.github.io/teaching/QMUL/QHP4701/

***Other Book**: Ref: Python Data Science Handbook, 2nd Edition,*

Link: https://jakevdp.github.io/PythonDataScienceHandbook/

Queen Mary

University of London